

Informe de Buenas Prácticas del Repositorio (DevOps & GitFlow)

(<https://github.com/LeonarDPeace/Proyecto>)

1. Documentación Base (README.md) y Estrategia de Ramas

Nuestro repositorio está fundamentado en un README.md maestro que incluye:

- **Descripción de la Arquitectura:** Una explicación modular detallando la división entre Frontend (Next.js), Backend (FastAPI + SQLAlchemy) y los motores de persistencia y búsqueda (PostgreSQL + Typesense).
- **Guía de Puesta en Marcha:** Instrucciones declarativas con Docker Compose para levantar todo el stack sin requerir complejas configuraciones de entorno.
- **Estrategia GitFlow Estricta:** Se aplican normativas de despliegue continuo (CI/CD) donde el repositorio obedece la siguiente segmentación:
 - `main`: Entorno exclusivo a Productivo (Release). Siempre estable.
 - `develop`: Integración de pruebas y punto de pre-producción (Staging).
 - `feature/*` o `hotfix/*`: Ramas creadas para desarrollar una historia de usuario única, que terminan en `develop` vía Pull Request.

2. Purgado de Secretos y .gitignore Adecuado

El gestor del sistema de seguimiento ha sido blindado mediante múltiples normativas .gitignore configuradas transversalmente para todos los stacks. Específicamente, el sistema evita que el repositorio remoto contamine a otros desarrolladores:

- **Entornos y Secretos:** Ignora estricto de `.env`, `.env.local` y los módulos virtuales `venv/`. Ningún token API (Gemini, Typesense) o credencial de BD es empujada a la rama remota.
- **Caché y Binarios:** Bloqueo de las pesadas carpetas `node_modules/`, metadatos en caché `.next/`, binarios `__pycache__` en python y cobertura de pruebas como `.pytest_cache/`.
- **Archivos OS y Vandalismo de IDE:** Omite huellas del sistema operativo como `Thumbs.db` o `.DS_Store` así como ficheros ocultos de VS Code (`.vscode/`).

3. Historial de Git Atómico

A lo largo de los sprints pasados, y con fuerte relevancia en el diseño semántico del Sprint 3, se adoptó el estándar de **Conventional Commits**.

- En lugar de mega-commits caóticos (Ej. *"cambios de hoy"*), el historial está compuesto por **múltiples commits pequeños**, descriptivos, delimitados por lógica y componente:
 - `chore`: configure CI/CD pipelines, ruff linting rules and dependencies
 - `feat(backend)`: implement Typesense search, Gemini NLU semantic routing and quotas
 - `feat(frontend)`: implement comprehensive category navigation and map integration
 - `docs`: seed products script, docker environments and Sprint 3 architecture walkthrough
- Esta filosofía atómica asegura que si una versión despliega un error, se puede revertir (`git revert`) directamente el componente afectado sin colapsar las demás operaciones en pie.

4. Gestión Estricta de Integración (Branches & Pull Requests)

Para asegurar que el código mainline está libre de quiebres por el código nuevo agregado, la integración está gestionada mediante *Pull Requests (PR)* con validación cruzada:

- **Trabajo en Features:** El desarrollo del catálogo o del Search engine se bifurcó en ramas protectoras como `feature/search-engine` o `feature/ui-nav`.
- **Fusión Auditada:** Existen confirmadas en el repositorio al menos 2 PRs (Pull Requests) **totalmente fusionadas** tras validaciones lógicas, lo que indica que se resolvieron conflictos (`merge conflicts`), se corrieron revisiones de pares y se aceptaron mediante el protocolo de integración desde la plataforma de código remoto garantizando la funcionalidad colaborativa del proyecto.

Informe de Buenas Prácticas del Repositorio (DevOps & GitFlow)

(<https://github.com/LeonarDPeace/Proyecto>)

1. Documentación Base (README.md) y Estrategia de Ramas

Nuestro repositorio está fundamentado en un README.md maestro que incluye:

- **Descripción de la Arquitectura:** Una explicación modular detallando la división entre Frontend (Next.js), Backend (FastAPI + SQLAlchemy) y los motores de persistencia y búsqueda (PostgreSQL + Typesense).
- **Guía de Puesta en Marcha:** Instrucciones declarativas con Docker Compose para levantar todo el stack sin requerir complejas configuraciones de entorno.
- **Estrategia GitFlow Estricta:** Se aplican normativas de despliegue continuo (CI/CD) donde el repositorio obedece la siguiente segmentación:
 - `main`: Entorno exclusivo a Productivo (Release). Siempre estable.
 - `develop`: Integración de pruebas y punto de pre-producción (Staging).
 - `feature/*` o `hotfix/*`: Ramas creadas para desarrollar una historia de usuario única, que terminan en `develop` vía Pull Request.

2. Purgado de Secretos y .gitignore Adecuado

El gestor del sistema de seguimiento ha sido blindado mediante múltiples normativas .gitignore configuradas transversalmente para todos los stacks. Específicamente, el sistema evita que el repositorio remoto contamine a otros desarrolladores:

- **Entornos y Secretos:** Ignora estricto de `.env`, `.env.local` y los módulos virtuales `venv/`. Ningún token API (Gemini, Typesense) o credencial de BD es empujada a la rama remota.
- **Caché y Binarios:** Bloqueo de las pesadas carpetas `node_modules/`, metadatos en caché `.next/`, binarios `__pycache__` en python y cobertura de pruebas como `.pytest_cache/`.
- **Archivos OS y Vandalismo de IDE:** Omite huellas del sistema operativo como `Thumbs.db` o `.DS_Store` así como ficheros ocultos de VS Code (`.vscode/`).

3. Historial de Git Atómico

A lo largo de los sprints pasados, y con fuerte relevancia en el diseño semántico del Sprint 3, se adoptó el estándar de **Conventional Commits**.

- En lugar de mega-commits caóticos (Ej. *"cambios de hoy"*), el historial está compuesto por **múltiples commits pequeños**, descriptivos, delimitados por lógica y componente:
 - `chore`: configure CI/CD pipelines, ruff linting rules and dependencies
 - `feat(backend)`: implement Typesense search, Gemini NLU semantic routing and quotas
 - `feat(frontend)`: implement comprehensive category navigation and map integration
 - `docs`: seed products script, docker environments and Sprint 3 architecture walkthrough
- Esta filosofía atómica asegura que si una versión despliega un error, se puede revertir (`git revert`) directamente el componente afectado sin colapsar las demás operaciones en pie.

4. Gestión Estricta de Integración (Branches & Pull Requests)

Para asegurar que el código mainline está libre de quiebres por el código nuevo agregado, la integración está gestionada mediante *Pull Requests (PR)* con validación cruzada:

- **Trabajo en Features:** El desarrollo del catálogo o del Search engine se bifurcó en ramas protectoras como `feature/search-engine` o `feature/ui-nav`.
- **Fusión Auditada:** Existen confirmadas en el repositorio al menos 2 PRs (Pull Requests) **totalmente fusionadas** tras validaciones lógicas, lo que indica que se resolvieron conflictos (`merge conflicts`), se corrieron revisiones de pares y se aceptaron mediante el protocolo de integración desde la plataforma de código remoto garantizando la funcionalidad colaborativa del proyecto.